

Catalogizer

**Gestión avanzada de colecciones multimedia multiprotocolo
— detecta, cataloga y enriquece todo lo que posees.**

Resumen

Catalogizer es un sistema autohospedable de gestión de colecciones multimedia que detecta, categoriza y organiza automáticamente medios en SMB, FTP, NFS, WebDAV y sistemas de archivos locales, con monitoreo en tiempo real, almacenamiento cifrado, enriquecimiento de metadatos externos y una moderna interfaz React respaldada por un Go de alto rendimiento y un API API.

Descripción breve

Un gestor de bibliotecas multimedia de nivel profesional y multiprotocolo. Un Go/Gin REST API detecta más de 50 tipos de medios en fuentes SMB/FTP/NFS/WebDAV/locales, los enriquece con datos de TMDB/IMDB/MusicBrainz/Steam y otros, y ofrece una aplicación web React en tiempo real sobre una base de datos cifrada SQLCipher.

Descripción detallada

La mayoría de los gestores de medios exigen que cedas primero: consolida todo en un solo disco, en un formato único, de un solo tipo, y solo entonces te ayudarán. Catalogizer parte de la premisa opuesta: tu colección ya está donde está, dispersa en recursos compartidos de NAS y protocolos que nunca se pondrán de acuerdo, y se adapta a ello. Habla los protocolos que ya usa el almacenamiento —SMB/CIFS, FTP/FTPS, NFS, WebDAV y sistemas de archivos locales— tras una abstracción unificada de cliente, de modo que un recurso compartido de Windows, un archivo FTP y un montaje WebDAV se ven idénticos para las capas superiores y pueden mezclarse, intercambiarse o retirarse sin tocar el código de la aplicación. Un backend Go (Gin REST API) supervisa continuamente esas fuentes, detecta y clasifica más de

50 tipos de medios (películas, series, música, juegos, software, documentales) a medida que aparecen los archivos, y enriquece cada elemento con datos de una pila de proveedores externos — TMDb, IMDb, TVDB, MusicBrainz, Spotify, Steam y otros—, transformando un simple nombre de archivo en una entrada de catálogo completamente atribuida, con carátulas, reparto y metadatos. Los resultados se transmiten al frontend TypeScript React mediante WebSockets, por lo que la biblioteca se actualiza en vivo durante la ingesta, sin necesidad de refrescar manualmente, y cada byte de metadatos se almacena en una base de datos cifrada SQLCipher protegida por autenticación basada en JWT y roles.

Mientras que la mayoría de los catalogadores fallan silenciosamente cuando un recurso compartido se cae, Catalogizer está diseñado para seguir siendo útil durante la interrupción. Un fallo temporal en SMB se mitiga con reconexiones de retroceso exponencial, un disyuntor que evita saturar un host inactivo, monitoreo continuo de salud y una caché de metadatos sin conexión que sigue respondiendo a las solicitudes de los usuarios con el último estado conocido válido. La diferencia está entre *“la aplicación entera se cae porque un NAS se reinició”* y *“una fuente está degradada, pero todo lo demás funciona”*. Más allá del catálogo, también actúa como herramienta operativa para la colección: análisis de tendencias de crecimiento y seguimiento de calidad/versiones, generación de informes profesionales PDF, servicio de conversión PDF a imagen/texto/HTML, exportación/importación de favoritos (JSON/CSV) y sincronización en la nube con S3, Google Cloud Storage o carpetas locales. Y no es un monolito que resulta ser grande, sino que está deliberadamente ensamblado a partir de 21 submódulos Go reutilizables `digital.vasic.*` más paquetes cliente TypeScript, cada uno probado y versionado de forma independiente. Así, los mismos componentes de autenticación, sistema de archivos, transmisión y observabilidad —ya probados en batalla— que impulsan Catalogizer también alimentan al resto de la familia de productos. La garantía de calidad no se basa en autodeclaraciones: el marco de *Challenges* y HelixQA someten cada capacidad anunciada a una verificación antifraude respaldada por evidencia.

Por qué lo creamos

Los gestores de medios existentes parten de un único backend de almacenamiento y un único tipo de medio. Las colecciones reales abarcan múltiples recursos compartidos en NAS y protocolos, se degradan cuando un recurso falla y combinan películas, música, juegos y software. Catalogizer se diseñó para tratar todos los

protocolos por igual, sobrevivir a almacenamientos en red inestables y ofrecer un catálogo único, enriquecido y cifrado que lo abarca todo.

Por qué cambia las reglas del juego

Consolida en un solo paquete autoalojable y cifrado lo que normalmente requiere una pila de herramientas independientes: ingesta agnóstica de protocolos que trata cada backend de almacenamiento con equidad, resiliencia que mantiene el catálogo operativo durante caídas de almacenamiento en lugar de colapsar con ellas, y enriquecimiento multi-proveedor que transforma archivos crudos en una biblioteca navegable y con metadatos. El beneficio de la arquitectura modular es acumulativo: una corrección de seguridad en el cliente del sistema de archivos o un nuevo complemento de proveedor se implementa una sola vez y mejora a todos los consumidores, de modo que Catalogizer mejora de forma constante a medida que lo hace el ecosistema que lo rodea. En resumen, marca la diferencia entre un índice de medios y un *sistema* de medios: uno que es tuyo, que sobrevive a infraestructuras inestables y cuyos componentes internos están probados, no solo prometidos.

Qué lo hace innovador

- Cliente unificado de sistema de archivos multi-protocolo (SMB/FTP/NFS/WebDAV/local) tras una única interfaz.
- Caché sin conexión + disyuntor para que el catálogo siga siendo utilizable durante caídas de almacenamiento.
- Extracción completa en 21 submódulos reutilizables `digital.vasic.*` de Go y módulos cliente en TS.
- Catálogo cifrado en reposo (SQLCipher) con sincronización en tiempo real WebSocket con la interfaz de usuario.
- Control de calidad basado en evidencia mediante el marco Challenges y la integración con HelixQA.

Desafíos y soluciones

- **Almacenamiento en red inestable:** resuelto con retroceso exponencial, disyuntor, comprobaciones de estado y una caché sin conexión con política de evicción que sirve metadatos almacenados cuando las fuentes no están disponibles.
- **Heterogeneidad de protocolos:** resuelto mediante la abstracción de cada protocolo tras un cliente común `digital.vasic.filesystem`, de modo que las capas superiores no dependen del protocolo.

- **Seguridad de los datos:** resuelto con cifrado en reposo SQLCipher, además de autenticación JWT/RBAC y middleware de saneamiento de solicitudes.
- **Mantenibilidad a escala:** resuelto extrayendo toda la lógica genérica a submódulos probados de forma independiente, en lugar de un monolito.

Pila tecnológica (por qué y cómo)

- **Go + Gin** — núcleo de alto rendimiento REST API (catalog-api); elegido por su concurrencia y rendimiento en cargas de trabajo de monitorización continua.
- **TypeScript + React + Tailwind (Vite)** — interfaz de usuario catalog-web responsiva con actualizaciones en tiempo real.
- **WebSockets** — sincronización en vivo de datos entre el backend y la interfaz de usuario.
- **SQLCipher (SQLite cifrado)** — almacenamiento de metadatos cifrado en reposo; soporte dual SQLite/PostgreSQL mediante `digital.vasic.database`.
- **Cientes SMB/FTP/NFS/WebDAV** — ingesta multi-protocolo a través de `digital.vasic.filesystem`.
- **APIs externas de metadatos (TMDB, IMDB, TVDB, MusicBrainz, Spotify, Steam)** — complementos de proveedores para enriquecimiento.
- **Prometheus + OpenTelemetry** — métricas y trazabilidad mediante `digital.vasic.observability`.
- **Docker / contenedor builder** — compilaciones reproducibles (Tauri/Rust canalizadas a través de catalogizer-builder).
- **Redis** — almacenamiento en caché y limitación de tasa mediante `digital.vasic.cache / ratelimiter`.
- **S3 / Google Cloud Storage** — sincronización en la nube y almacenamiento de puntos de control.